
Integration Manual

for MPC560XB PWM MCAL Driver

Document Number: IM13PWMASR4.0R1.0.1
Rev 1.2





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Supported Derivatives.....	8
2.2	Overview.....	8
2.3	About this Manual.....	9
2.4	Acronyms and Definitions.....	9
2.5	Reference List.....	10
Chapter 3		
Building the Driver		
3.1	Build Options.....	11
3.1.1	GHS Compiler/Linker/Assembler Options.....	11
3.1.2	DIAB Compiler/Linker/Assembler Options.....	13
3.1.3	CW Compiler/Linker/Assembler Options.....	14
3.1.4	HT Compiler/Linker/Assembler Options.....	16
3.2	Files required for Compilation.....	17
3.3	Setting up the Plug-ins.....	19
Chapter 4		
Function Calls to Module		
4.1	Function Calls during Start-up.....	21
4.2	Function Calls during Shutdown.....	21
4.3	Function Calls during Wake-up.....	21
Chapter 5		
Module Requirements		
5.1	Exclusive areas to be defined in BSW scheduler.....	23
5.2	Peripheral Hardware Requirements.....	23

Section number	Title	Page
5.3	ISR to Configure within OS - Dependencies	23
5.3.1	The following ISRs are used.....	23
5.3.2	Macros for Interrupts.....	25
5.4	ISR macro.....	25
5.5	Other AUTOSAR modules - dependencies.....	26

Chapter 6 Main API Requirements

6.1	Main functions calls within BSW.....	27
6.2	API Requirements.....	27
6.3	Calls to notification functions, callbacks, callouts.....	27

Chapter 7 Memory Allocation

7.1	Sections to be defined in MemMap.h.....	29
7.2	Linker command file.....	31

Chapter 8 Configuration Parameters

8.1	Configuration parameter considerations.....	33
-----	---	----

Chapter 9 Integration Steps

Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	17/02/2012	Subramanya M Naik	Changes for MPC560XB 0.9.0 BETA
1.1	17/05/2013	Manoj S R	Changes for MPC560XB 1.0.0 RTM
1.2	06/06/2014	Quyen Trong	Changes for MPC560XB 1.0.1 RTM



Chapter 2

Introduction

This Integration Manual describes the integration requirements for Autosar PWM Driver for Freescale Semiconductor's MPC560XB microcontrollers.

The roadmap for the document is as follows:

Building the Driver : This section gives a brief overview of the build procedure (compiler, linker options and source files) and Plugins setup.

Function Calls to Module : This section lists the various function calls to modules during Start-up, Shutdown and Wake-up.

Module Requirements : This section specifies the various module requirements related to

- Exclusive areas to be defined in BSW scheduler
- Peripheral Hardware Requirements
- Specific interface to other modules
- ISR to configure within OS
- Dependencies with other AUTOSAR modules

Main API Requirements : This section specifies the requirements related to the main Pwm API and gives a brief overview of the main functions calls within BSW scheduler, API_Name Requirements and calls to notification functions, callbacks, callouts.

Memory Allocation : This section describes the memory allocation requirements namely the sections to be defined in MemMap.h and the linker command file.

Configuration Parameters : This section covers the various Pre Compile, Link Time and Post Build time configuration parameters.

Integration Steps : This section describes in brief the steps for integrating Pwm module.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor :

Table 2-1. MPC560XB Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkl_lqfp100 mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
-------------------------	---

All of the above microcontroller devices are collectively named as MPC560XB.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.

- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
BSW	Basic Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
PWM	Pulse Width Modulation
ISR	Interrupt Service Routine
OS	Operating System
MCU	Microcontroller Unit
GUI	Graphical User Interface
API	Application Programming Interface
PB Variant	Post Build Variant
PC Variant	Pre Compile Variant

2.5 Reference List

Table 2-3. Reference List

Title	Version
AUTOSAR PWM Driver Software Specification Document	V2.5.0
MPC5607B Reference Manual	Rev. 7.2, May 2012
MPC5604B Reference Manual	Rev. 8.2, September 2013
5606B Reference Manual	Rev. 2, May 2014
5602D Reference Manual	Rev. 4.2, Dec 2013
Bolero 1.5M cut 2.2 Errata Sheet (MPC5607B_1M03Y)	Rev. 12Dec2013
Bolero 512k cut 2.3 Errata Sheet (MPC5604B_2M27V)	Rev. 26 FEB 2014
Bolero 1M cut 1.0 Errata Sheet (MPC5606B_0N13E)	Rev. , 12 DEC 2013
Bolero 256k cut 1.1 Errata Sheet (MPC5602D_1M18Y)	Rev. , 12 DEC 2013

Chapter 3

Building the Driver

This section describes the source files and various compiler, linker options used for building the Autosar Pwm driver for Freescale Semiconductor's MPC560XB microcontrollers. It also explains the Plugins setup procedure.

3.1 Build Options

The driver files are compiled using Green Hills MULTI v6.1.4 , Windriver DIAB Rel DIAB_5_9_2_0-FCS_20120725_234511, HighTec v4.6.4.0, Compiler gcc v4.6.3 and Freescale CodeWarrior for MPC5xxx v2.10 - compiler Version 4.3 build 224 . The compiler, linker flags used for building the driver are explained below:

Note:

The Pwm_TS_T2D13M10I1R0 is composed as follow:
TS_T<Target_Id>D<Derivative_Id>M<SW_Version_Major>I<SW_Version_Minor>R0
(i.e. Target_Id = 2 identifies PowerPC architecture and Derivative_Id = 13 identifies the MPC560XB derivatives)

3.1.1 GHS Compiler/Linker/Assembler Options

Table 3-1. Compiler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-ansi	Specifies ANSI C with extensions. This mode extends the ANSI X3.159-1989 standard with certain useful and compatible constructs.
-noSPE	Disables the use of SPE and vector floating point instructions by the compiler.
-Ospace	Optimize for size
-sda=0	Enables the Small Data Area optimization with a threshold of 0.
-vle	Enables VLE code generation

Table continues on the next page...

Table 3-1. Compiler Options (continued)

Option	Description
-dual_debug	Enables the generation of DWARF, COFF, or BSD debugging information in the object file
-G	Generates source level debugging information and allows procedure call from debugger's command line.
--no_exceptions	Disables support for exception handling
-Wundef	Generates warnings for undefined symbols in preprocessor expressions
-Wimplicit-int	Issues a warning if the return type of a function is not declared before it is called
-Wshadow	Issues a warning if the declaration of a local variable shadows the declaration of a variable of the same name declared at the global scope, or at an outer scope
-Wtrigraphs	Issues a warning for any use of trigraphs
--prototype_errors	Generates errors when functions referenced or called have no prototype
--incorrect_pragma_warnings	Valid #pragma directives with wrong syntax are treated as warnings
-noslashcomment	C++ like comments will generate a compilation error
-preprocess_assembly_files	Preprocesses assembly files
--short_enum	Store enumerations in the smallest possible type
--no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup.
-c	Produces an object file (called input-file.o) for each source file.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.
-DDISABLE_MCAL_INTERMODULE_ASR_CHECK	-D defines a preprocessor symbol to disable the inter-module version check for AR_RELEASE versions. DISABLE_MCAL_INTERMODULE_ASR_CHECK: By default in the package, drivers are compiled to perform the inter-module version check as per Autosar BSW004. When the inter-module version check needs to be disabled then the DISABLE_MCAL_INTERMODULE_ASR_CHECK global define must be added to the list of compiler options.
-DGHS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the GHS preprocessor symbol.

Table 3-2. Assembler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb

Table 3-3. Linker Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-nostartfiles	Do not use Start files.
-vle	Enables VLE code generation
-linker_warnings	Display linker warnings

3.1.2 DIAB Compiler/Linker/Assembler Options

Table 3-4. Compiler Options

Option	Description
-Xdialect-ansi	Follow the ANSI C standard with some additions
-XO	Enables extra optimizations to produce highly optimized code
-g	Generate symbolic debugger information and disable some optimizations.
-Xsize-opt	Optimize for size rather than speed when there is a choice
-Xsmall-data=0	Set Size Limit for "small data" Variables to zero.
-Xaddr-sconst=0x11	Specify addressing for constant static and global variables with size less than or equal to -Xsmall-const to far-absolute.
-Xaddr-sdata=0x11	Specify addressing for non-constant static and global variables with size less than or equal to -Xsmall-data in size to far-absolute.
-Xno-common	Disable use of the "COMMON" feature so that the compiler or assembler will allocate each uninitialized public variable in the .bss section for the module defining it, and the linker will require exactly one definition of each public variable
-Xnested-interrupts	Allow nested interrupts
-Xdebug-dwarf2	Generate symbolic debug information in dwarf2 format
-Xdebug-local-all	Force generation of type information for all local variables
-Xdebug-local-cie	Create common information entry per module
-Xdebug-struct-all	Force generation of type information for all typedefs, struct, union and class types
-Xforce-declarations	Generates warnings if a function is used without a previous declaration
-ee1481	Generate an error when the function was used before it has been declared
-Xmacro-undefined-warn	Generates a warning when an undefined macro name occurs in a #if preprocessor directive
-W:as;,-l	Pass the option "-l" (lower case letter L) to the assembler to get an assembler listing file
-Wa,-Xisa-vle	Instruct the assembler to expect and assemble VLE (Variable Length Encoding) instructions rather than BookE instructions.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DDIAB	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the DIAB preprocessor symbol.
-Xforce-prototypes	-Generate warnings if a function is used without a previous prototype declaration.

Table continues on the next page...

Table 3-4. Compiler Options (continued)

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries.

Table 3-5. Assembler Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-g	Dump the symbols in the global symbol table in each archive file.
-Xisa-vle	Expect and assemble VLE (Variable Length Encoding) instructions rather than Book E instructions. The default code section is named .text_vle instead of .text, and the default code section fill "character" is set to 0x44444444 instead of 0. The .text_vle code section will have ELF section header flags marking it as VLE code, not Book E code.

Table 3-6. Linker Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-Xelf	Generates ELF object format for output file
-m6	Generates a detailed link map and cross reference table
-lc	Specifies to linker to search for libc.a
-Xlibc-old	Enables usage of legacy (pre-release 5.6) libraries

3.1.3 CW Compiler/Linker/Assembler Options

Table 3-7. Compiler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-lang c	Expects source code to conform to the language specified by the ISO/IEC 9899-1990 ("C90") standard
-opt all	This option is selected all optimization (the same as -opt speed,level=4,intrinsics,noframe)
-common off	Disables moving uninitialized data into a common section
-sdatathreshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small data. (The linker stores small data items in the Small Data address space. The compiler can generate faster code to access this data.)

Table continues on the next page...

Table 3-7. Compiler Options (continued)

Option	Description
-sdata2threshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small constant data. (The linker stores small constant data items in the Small Constant Data address space.)
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-use_lmw_stmw on	Enables the use of multiple load and store instructions for function prologues and epilogues
-ppc_asm_to_vle	Converts regular Power Architecture assembler mnemonics to equivalent VLE (Variable Length Encoded) assembler mnemonics in the inline assembler
-Cpp_exceptions off	When on, generates executable code for C++ exceptions. When off, generates smaller, faster executable code
-func_align 4	Specifies alignment of functions in executable code
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format
-w on	Turns on most warning messages
-r	Compiler should expect function prototypes
-w undefmacro	Issues warning messages on the use of undefined macros in #if and #elif conditionals
-char unsigned	Controls the default sign of the char data type: char data items are unsigned
-nosyspath	Performs a search of both the user and system paths, treating #include statements of the form #include xyz the same as the form #include "xyz"
-fp none	No floating point code generation
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-8. Assembler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format.
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-9. Linker Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-code_merging all	Removes duplicated functions to reduce object code size
-far_near_addressing	Simplifies address computations to reduce object code size and improve performance
-vle_enhance_merging	Removes duplicated functions that are called by functions that use VLE instructions to reduce object code size
-listdwarf	DWARF debugging information in the linker's map file
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-char unsigned	Controls the default sign of the char data type: char data items are unsigned.

3.1.4 HT Compiler/Linker/Assembler Options

Table 3-10. Compiler Options

Option	Description
-Os	Optimize for size. '-Os' enables all '-O2' optimizations that do not typically increase code size. It also performs further optimizations designed to reduce code size.
-msdata=eabi	On System V.4 and embedded PowerPC systems, put small initialized const global and static data in the '.sdata2' section, which is pointed to by register r2. Put small initialized non-const global and static data in the '.sdata' section, which is pointed to by register r13. Put small uninitialized global and static data in the '.sbss' section, which is adjacent to the '.sdata' section.
-G1000	On embedded PowerPC systems, put global and static items less than or equal to num bytes into the small data or bss sections instead of the normal data or bss section.
-fshort-double	Use the same size for double as for float.
-mno-spe	This option used to disable spe extension (Like -noSPE on ghs compiler).
-mregnames	On System V.4 and embedded PowerPC systems do (do not) emit register names in the assembly language output using symbolic forms
-g3	Request debugging information of Level 3. Includes extra information, such as all the macro definitions present in the program.
-Wall	This enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros.
-mvle	Enable support of VLE
--mcpu=z425n3	Support of e200 Z425n3 core
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be

Table 3-10. Compiler Options

Option	Description
	added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.

Table 3-11. Assembler Options

Option	Description
-c	Compile or assemble the source files, but do not link. The linking stage simply is not done. The ultimate output is in the form of an object file for each source file.
-Wa,--gdwarf2	Produce debugging information in DWARF format.
-msdata=eabi	Put small initialized const global and static data in the .sdata2 section, which is pointed to by register r2. Put small initialized non-const global and static data in the .sdata section, which is pointed to by register r13. Put small uninitialized global and static data in the .sbss section, which is adjacent to the .sdata section.
-mregnames	Do emit register names in the assembly language output using symbolic forms.
-mno-spe	This option used to disable spe extension (Like --noSPE on ghs compiler).
-mcpu=z425n3	Support of e200 z425n3 core

Table 3-12. Linker Options

Option	Description
-N	Set the text and data sections to be readable and writable. Also, do not page-align the data segment. If the output format supports Unix style magic numbers, mark the output as OMAGIC.
--no-undefined	Normally when a symbol has an undefined version, the linker will ignore it. This option disallows symbols with undefined version and a fatal error will be issued instead.
--warn-common	Normally ppc-vle-ld will give a warning if it finds an incompatible library during a library search. This option silences the warning.
--error-unresolved-symbols	This restores the linker's default behaviour of generating errors when it is reporting unresolved symbols.
-nocrt0	library crt0 from compiler library not used, it uses custom crt0 file
-nostartfiles	no start file from compiler library is used
-Wl	If the linker is being invoked indirectly, via a compiler driver (e.g. 'gcc') then all the linker command line options should be prefixed by '-Wl,'Used in combination with -no-warn-flags, --no-undefined, --warn-common and --error-unresolved-symbols
--no-warn-flags	Used in combination with -Wl disable linker warnings. See previous
-T	The -T option specifies the name of your own script ie sprig.link. The -c is a synonym of the -T option

3.2 Files required for Compilation

This section describes the include files required to compile, assemble (if assembler code) and link the Autosar PWM driver for Freescale Semiconductor's MPC560XB microcontrollers.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJOR_VERSION and AR_MINOR_VERSION, i.e. only files with the same Autosar major and minor versions can be compiled.

Pwm Files:

Table 3-13. Include Files

..\Pwm_TS_T2D13M10I1R0\include\	Pwm.h
	Pwm_eMIOS_LLD.h
	Pwm_eMIOS_LLD_CfgEx.h
	Pwm_LLD.h
	Pwm_NonASR.h
	Reg_eSys_eMIOS.h
	Reg_eSys_eMIOS_CfgEx.h

Table 3-14. Source Files

..\Pwm_TS_T2D13M10I1R0\src\	Pwm.c
	Pwm_eMIOS_Irq.c
	Pwm_eMIOS_LLD.c
	Pwm_NonASR.c
	Pwm_Reg_eSys_eMIOS.c

Pwm_Cfg.c (For PC Variant) - This file should be generated by the user using a configuration tool for compilation

Pwm_PBcfg.c (For PB Variant) - This file should be generated by the user using a configuration tool for compilation

Pwm_Cfg.h - This file should be generated by the user using a configuration tool for compilation

Other include files:

Table 3-15. Files from Base folder:

..\Base_TS_T2D13M10I1R0\include\	Cer.h
----------------------------------	-------

Table continues on the next page...

Table 3-15. Files from Base folder: (continued)

	Compiler.h
	Compiler_Cfg.h
	ComStack_Cfg.h
	ComStack_Types.h
	Mcal.h
	MemMap.h
	Platform_Types.h
	Reg_eSys.h
	Reg_Macros.h
..\Base_TS_T2D13M10I1R0\generate_PC\include\	Std_Types.h
	Soc_Ips.h
	modules.h

Table 3-16. Files from Dem folder:

..\Dem_TS_T2D13M10I1R0\include\	Dem.h
	Dem_Types.h
..\Dem_TS_T2D13M10I1R0\generate_PC\include\	Dem_IntErrId.h

Table 3-17. Files from Det folder:

..\Det_TS_T2D13M10I1R0\include\	Det.h
---------------------------------	-------

Table 3-18. Files from Rte folder:

..\Rte_TS_T2D13M10I1R0\include\	SchM_Pwm.h
---------------------------------	------------

3.3 Setting up the Plug-ins

All the Autosar MCAL drivers for MPC560XB were designed to be configured using Tresos® Studio configuration and code generation tool from tresos Elektrobit Tresos Release 14.2.1 (b140128-1223).

Location of various files inside the plugin folder is explained below.

Module Parameter Definition File:

..\Pwm_TS_T2D13M10I1R0\config\Pwm.xdm

Code Generation Templates for Pre-Compile time configuration parameters:

..\Pwm_TS_T2D13M10I1R0\generate_PC\src\Pwm_Cfg.c

..\Pwm_TS_T2D13M10I1R0\generate_PC\include\Pwm_Cfg.h

Code Generation Templates for Post-Build time configuration parameters:

..\Pwm_TS_T2D13M10I1R0\generate_PB\src\Pwm_PBcfg.c

Steps to generate configurations:

1. Copy the module folder Pwm_TS_T2D13M10I1R0 into the Tresos plugins folder.
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the Tresos GUI to modify configuration parameters values
4. Generate the Pre-Compile and Post-Build files.

Chapter 4

Function Calls to Module

The PWM module shall be initialized by Pwm_Init() service call during the start-up.

NOTE: Pins used for connection of PWM physical layer have to be properly assigned to the eMios module prior the PWM initialization.

4.1 Function Calls during Start-up

This driver does not need OS Support except for ISRs. Hence can be initialized either in STARTUP1 or STARTUP2 phase of EcuM initialization. This depends on the implementation, desired duration for STARTUP1 & Target hardware design. The API to be called is Pwm_Init(ConfigPtr).

4.2 Function Calls during Shutdown

None

4.3 Function Calls during Wake-up

None.

Chapter 5

Module Requirements

5.1 Exclusive areas to be defined in BSW scheduler

In the current implementation, PWM is using the services of Schedule Manager (SchM) for entering and exiting the critical regions.

Each exclusive area protects the Pwm channels from concurrent accesses.

The protected resources are the hw registers and the global variables associated with each Pwm channel.

The ISR critical regions must not block the other critical regions to avoid deadlocks. This is ensured by exiting the ISR critical region before calling the user notification functions.

In this case the SchM implementation must provide the required resources.

The following critical regions are used in the PWM driver:

5.2 Peripheral Hardware Requirements

For MPC560XB controllers eMios provide PWM functionality.

5.3 ISR to Configure within OS - Dependencies

5.3.1 The following ISRs are used

Table 5-1. Available ISRs

ISR Name	Hardware Interrupt vector
ISR(EMIOS_0_CH_0_CH_1_ISR)	141
ISR(EMIOS_0_CH_2_CH_3_ISR)	142
ISR(EMIOS_0_CH_4_CH_5_ISR)	143
ISR(EMIOS_0_CH_6_CH_7_ISR)	144
ISR(EMIOS_0_CH_8_CH_9_ISR)	145
ISR(EMIOS_0_CH_10_CH_11_ISR)	146
ISR(EMIOS_0_CH_12_CH_13_ISR)	147
ISR(EMIOS_0_CH_14_CH_15_ISR)	148
ISR(EMIOS_0_CH_16_CH_17_ISR)	149
ISR(EMIOS_0_CH_18_CH_19_ISR)	150
ISR(EMIOS_0_CH_20_CH_21_ISR)	151
ISR(EMIOS_0_CH_22_CH_23_ISR)	152
ISR(EMIOS_0_CH_24_CH_25_ISR)	153
ISR(EMIOS_0_CH_26_CH_27_ISR)	154
ISR(EMIOS_0_CH_28_CH_29_ISR)	155
ISR(EMIOS_0_CH_30_CH_31_ISR)	156
ISR(EMIOS_1_CH_0_CH_1_ISR)	157
ISR(EMIOS_1_CH_2_CH_3_ISR)	158
ISR(EMIOS_1_CH_4_CH_5_ISR)	159
ISR(EMIOS_1_CH_6_CH_7_ISR)	160
ISR(EMIOS_1_CH_8_CH_9_ISR)	161
ISR(EMIOS_1_CH_10_CH_11_ISR)	162
ISR(EMIOS_1_CH_12_CH_13_ISR)	163
ISR(EMIOS_1_CH_14_CH_15_ISR)	164
ISR(EMIOS_1_CH_16_CH_17_ISR)	165
ISR(EMIOS_1_CH_18_CH_19_ISR)	166
ISR(EMIOS_1_CH_20_CH_21_ISR)	167
ISR(EMIOS_1_CH_22_CH_23_ISR)	168
ISR(EMIOS_1_CH_24_CH_25_ISR)	169
ISR(EMIOS_1_CH_26_CH_27_ISR)	170
ISR(EMIOS_1_CH_28_CH_29_ISR)	171
ISR(EMIOS_1_CH_30_CH_31_ISR)	172

NOTE

For the ISR EMIOS_0/1_CH_X_CH_Y_ISR must be used in the corresponding vector locations. ISR EMIOS_0/1_CH_X_CH_Y_ISR is a common interrupt handler and is shared by GPT, ICU and PWM. Through configuration this common

handler will route the interrupt to the driver that uses this Emios unified channel. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.

5.3.2 Macros for Interrupts

For the ISR EMIOS_0/1_CH_X_ISR must be used in the corresponding vector locations. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.

5.4 ISR macro

MCAL drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions:

1. OS is not used - AUTOSAR_OS_NOT_USED is defined:

- If USE_SW_VECTOR_MODE is defined:

```
#define ISR(IsrName) void IsrName(void)
```

In this case, drivers' interrupt handlers are normal C functions and the prolog/epilog handle the context save and restore.

- If USE_SW_VECTOR_MODE is not defined:

```
#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)
```

In this case, drivers' interrupt handlers must save and restore the execution context.

2. Freescale Semiconductor OS is used – AUTOSAR_OS_NOT_USED is not defined

```
#define ISR(IsrName) void OS_isr_##IsrName()
```

In this case, OS is handling the execution context when an interrupt occurs. Drivers' interrupt handlers are normal C functions.

3. Other vendor's OS is used – AUTOSAR_OS_NOT_USED is not defined. Please refer to the OS documentation for description of the ISR macro.

Please refer to the OS documentation for description of the ISR macro.

5.5 Other AUTOSAR modules - dependencies

Development Error Tracer:

This module is necessary for enabling Development error detection. The API function used is Det_ReportError(). The activation / deactivation of Development error detection is configurable using the

'PwmDevErrorDetect' configuration parameter.

Diagnostic Event Manager:

This module is necessary for enabling Production error detection. The API function used is Dem_ReportErrorStatus ().

Mcu:

MCU module shall be initialized before using Pwm.

This module is required for setting the eMios global Pre-scalar value and clock.

Port:

PORT module shall configure the eMios channels which are to be used for the Pwm

Configuration dependency to other module:

Care must be used not to allocate the same eMios channels to other MCAL drivers (ICU/ GPT).

Chapter 6

Main API Requirements

6.1 Main functions calls within BSW

None

6.2 API Requirements

Not Applicable.

6.3 Calls to notification functions, callbacks, callouts

Call-back Notifications:

None.

User Notification:

The Pwm Driver provides a notification per channel that is called whenever the selected edges are generated.

The notifications can be configured as pointers to user defined functions.

If notification is not desired for a specific channel then 'NULL_PTR' shall be configured.

The syntax of this function is as follows:

```
void Pwm_Notification_<channel>( void )
```

An extern declaration of this function is available in Pwm_Cfg.c and Pwm_PBcfg.c. The function has to be implemented by the user.

Chapter 7

Memory Allocation

7.1 Sections to be defined in MemMap.h

For Post Build data:

```
#ifndef PWM_START_CONFIG_DATA_UNSPECIFIED
#undef PWM_START_CONFIG_DATA_UNSPECIFIED
#undef MEMMAP_ERROR

/*Memory Section for Post Build Data to be defined here. Example given in the next line*/

#pragma ghs section const=".pbPWM_cfg"
#endif

#ifndef PWM_STOP_CONFIG_DATA_UNSPECIFIED
#undef PWM_STOP_CONFIG_DATA_UNSPECIFIED
#undef MEMMAP_ERROR

/*End of section to be mentioned here. Example given in the next line.*/

#pragma ghs section
#endif
```

For Code:

```
#ifndef PWM_START_SEC_CODE
```

```
#undef PWM_START_SEC_CODE
#undef MEMMAP_ERROR
/*Memory Section for Code to be defined here.*/
#endif
```

```
#ifdef PWM_STOP_SEC_CODE
#undef PWM_STOP_SEC_CODE
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#endif
```

For Variables:

```
#ifdef PWM_START_SEC_VAR_NO_INIT_UNSPECIFIED
#undef PWM_START_SEC_VAR_NO_INIT_UNSPECIFIED
#undef MEMMAP_ERROR
/*Memory Section for Variables to be defined here.*/
#endif
```

```
#ifdef PWM_STOP_SEC_VAR_NO_INIT_UNSPECIFIED
#undef PWM_STOP_SEC_VAR_NO_INIT_UNSPECIFIED
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#endif
```

```
#ifdef PWM_START_SEC_VAR_NO_INIT_8
#undef PWM_START_SEC_VAR_NO_INIT_8
#undef MEMMAP_ERROR
```

```
/*Memory Section for Variables to be defined here.*/
```

```
#endif
```

```
#ifdef PWM_STOP_SEC_VAR_NO_INIT_8
```

```
#undef PWM_STOP_SEC_VAR_NO_INIT_8
```

```
#undef MEMMAP_ERROR
```

```
/*End of section to be mentioned here*/
```

```
#endif
```

For Constant data:

```
#ifdef PWM_START_SEC_CONST_32
```

```
#undef PWM_START_SEC_CONST_32
```

```
#undef MEMMAP_ERROR
```

```
/*Memory Section for Constants to be defined here.*/
```

```
#endif
```

```
#ifdef PWM_STOP_SEC_CONST_32
```

```
#undef PWM_STOP_SEC_CONST_32
```

```
#undef MEMMAP_ERROR
```

```
/*End of section to be mentioned here*/
```

```
#endif
```

7.2 Linker command file

Memory shall be allocated for every section defined in MemMap.h.

Chapter 8

Configuration Parameters

Configuration parameter class for Autosar PWM driver fall into the following variants as defined below:

8.1 Configuration parameter considerations

Configuration parameter class for Autosar Pwm driver fall into the following variants as defined below:

Table 8-1. Configuration Parameters

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
PwmGeneral	PwmDevErrorDetect	VariantPC or VariantPB	PreCompile
	PwmChangeRegisterA	VariantPC or VariantPB	This parameter is not used in the current implementation.
	PwmDutycycleUpdatedEndperiod	VariantPC or VariantPB	Pre Compile
	PwmNotificationSupported	VariantPC or VariantPB	This parameter is not used in the current implementation.
	PwmPeriodUpdatedEndperiod	VariantPC or VariantPB	This parameter is not used in the current implementation.
	PwmIndex	VariantPC or VariantPB	Pre Compile
	PwmClockFromMCU	VariantPC or VariantPB	Pre Compile
	PwmCpuClockRef	VariantPC or VariantPB	Pre Compile
	PwmEmiosClockValue	VariantPC or VariantPB	Pre Compile
	PwmGenerateClockTreeDebugInfo	VariantPC or VariantPB	Pre Compile
	PWM_E_UNEXPECTED_IRQ	VariantPC or VariantPB	Pre Compile
PwmNonAUTOSAR	PwmEnableDualClockMode	Pre Compile parameter for all Variants of Configuration	Pre Compile
	PwmSetChannelOutputApi	VariantPC or VariantPB	Pre Compile
	PwmDisableDemReportErrorStatus	VariantPC or VariantPB	Pre Compile
	MultiPwmChannelSynch	VariantPC or VariantPB	Pre Compile

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

	PwmSetCounterBusApi	VariantPC or VariantPB	Pre Compile
	PwmSetTriggerDelay	VariantPC or VariantPB	Pre Compile
	PwmGetChannelStateApi	VariantPC or VariantPB	Pre Compile
PwmConfigurationOfOptApi Services	PwmDelInitApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	PwmGetOutputState	Pre Compile parameter for all Variants of Configuration	Pre Compile
	PwmSetDutyCycle	Pre Compile parameter for all Variants of Configuration	Pre Compile
	PwmSetOutputTogle	Pre Compile parameter for all Variants of Configuration	Pre Compile
	PwmSetPeriodAndDuty	Pre Compile parameter for all Variants of Configuration	Pre Compile
	PwmVersionInfoApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
PwmChannelConfigSet	PwmChannelClass	VariantPC or VariantPB	Post Build
PwmChannel	PwmChannelId	VariantPC or VariantPB	Pre Compile
	PwmHwChannel	VariantPC or VariantPB	Pre Compile
	PwmPolarity	VariantPC or VariantPB	Post Build
	PwmPeriodDefaultUnits	VariantPC or VariantPB	Post Build
	PwmPeriodDefault	VariantPC or VariantPB	Post Build
	PwmDutycycleDefault	VariantPC or VariantPB	Post Build
	PwmIdleState	VariantPC or VariantPB	Post Build
	PwmNotification	VariantPC or VariantPB	Post Build
	PwmModeSelect	VariantPC or VariantPB	Post Build
	EmiosUnifiedChannelBusSelect	VariantPC or VariantPB	Post Build
	PwmPrescaler	VariantPC or VariantPB	Post Build
	PwmPrescaler_Alternate	VariantPC or VariantPB	Post Build
	PwmOffset	VariantPC or VariantPB	Post Build
	PwmTriggerDelay	VariantPC or VariantPB	Post Build
	Pwm_Deadtime	VariantPC or VariantPB	Post Build
	OffsetDelayAdjust	VariantPC or VariantPB	Post Build
	PwmFreezeEnable	VariantPC or VariantPB	Post Build

Chapter 9

Integration Steps

This section gives a brief overview of the steps needed for integrating Pwm:

- Generate the required Pwm configurations. For more details refer to section [Calls to notification functions, callbacks, callouts](#).
- Allocate proper memory sections in MemMap.h and linker command file. For more details refer to section [Sections to be defined in MemMap.h](#).
- Make sure all include files for compilation are as per section [ISR macro](#).
- Map the ISRs to their vector locations. For more details refer to section [Peripheral Hardware Requirements](#).
- Compile & build the Pwm with all the dependent modules. For more details refer to section [Building the Driver](#) and [ISR to Configure within OS - Dependencies](#).



How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, AltiVec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number IM13PWWMASR4.0R1.0.1
Revision 1.2